

Important AIGPX Technical Specs

Simulation Environment

From the Technical specs for the competition:

The environment consists of a

- start *gate*, sequential race gates, and a finish gate which are visually distinctive and consistent.
- Beyond the first qualifier, there will be horizontal and vertical obstacles, boundary elements, and terrain + environmental structures

Simulation implements rigid-body drone model with

- Thrust
- Drag
- Gravity
- Collision physics

Physics refresh rate: 120Hz

The simulator uses a local Cartesian coordinate system internally, so no geographic coordinates and no GPS. Absolute position isn't exposed. Only doing distance calculations etc. via reference points relative to one another e.g. the start position.

Visually the simulation provides a forward-facing first-person camera and a visual environment with the objects specified above. Essentially static 3D objects and dynamic lighting.

Course geometry is identical for contestants, physics parameters are identical, and environmental conditions are deterministic.

Drone/Environment Specs

Drone Chassis

- Width: 280mm
- Length: 280mm
- Height: 160mm

Gate dimension

- Width: 2700mm
- Height: 2700mm
- Depth: 260mm

There's a 1500x1500mm inner gate that is 260mm deep to pass through with the drone based on the picture

Gate inner square boundaries

- Width: 1500mm
- Height: 1500mm
- Depth: 260mm

Given two coordinate frames during the competition that use Mavlink2 NED convention.

1. MAV_FRAME_BODY_NED - The origin is the vehicle itself. X points forward, Y points right, Z points down.
2. MAV_FRAME_LOCAL_NED - Origin (0,0,0) fixed physical point on the ground (usually where drone armed)

Does the MAV_FRAME_BODY_NED change over time? How can we calculate distance from the starting point to figure out where we are in space? 3D distance formula is:

- $\text{distance} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}$

Vision/Camera Specs

Body to Camera mapping

- Camera and body frame the same origin.
- The camera is tilted 20 degrees upward.

Specific note that NED coordinates might require that we rotate the camera frame into the camera coordinate convention of the image processing library we use.

Body to IMU mapping

"Is the identity map" so I think they're equivalent? What is IMU?

Camera intrinsics

- Standard pinhole camera model without lens distortion
- Resolution: 640px by 360px
- [cx, cy]: [320px, 180px]
- [fx, fy]: [320, 320]
- VFoV: 90 degrees

What is (cx, cy) vs (fx, fy)?

Communication Protocol (mavlink)

Intended for MAV (basically miniature unmanned aerial vehicles) communications between the control station and the drone.

https://github.com/mavlink/c_library_v2

Simulator connects via UDP to contestant control software which we're building through "MAVSDK-compatible interfaces"

Supported MAVLink messages

https://mavlink.io/en/guide/mavlink_2.html

Message	Direction	Purpose
HEARTBEAT	Sim → Client	Connection Status (e.g. ping)
ATTITUDE	Sim → Client	Vehicle "attitude"
HIGHRES_IMU	Sim → Client	Vehicle status
SET_POSITION_TARGET_LOCAL_NED	Client → Sim	Control interface
SET_ATTITUDE_TARGET	Client → Sim	Control interface
TIMESYNC	Sim → Client	Timing
HIGHRES_IMU (duplicated?)	Sim → Client	Measurements

Packet structure for UDP datagram stream

Field	Type	Size	Description
frame_id	uint32	4B	Unique sequence ID for image frame
chunk_id	uint16	2B	Index of packet within the frame (0 to total_chunks - 1)
total_chunks	uint16	2B	Total number of packets to complete frame
jpeg_size	uint32	4B	Total size of final reconstructed JPEG in B
payload_size	uint32	4B	Size of JPEG data slice in this packet
sim_time_ns	uint64	8B	Simulation epoch timestamp in nanoseconds

Timings

- Physics refresh: 120Hz
- Command rate: <100Hz
- Minimum heartbeat rate: 2Hz

Telemetry

- Vehicle attitude
- Orientation
- Linear velocities
- System status flags

Vision Stream

- 30Hz (30fps) 640px by 360px
- Transported on UDP by port 5600 little-endian 24Bheader

SITL (software in the loop) bridge

- Simulator provides UDP SITL bridge which enables external AI controllers to exchange telemetry and control commands

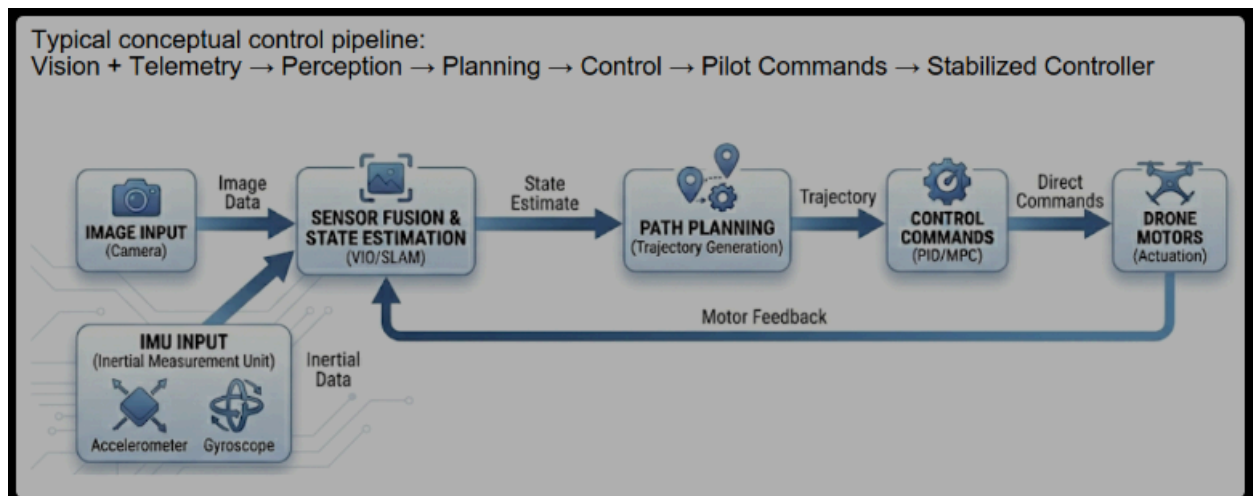
Contestant Software Environment

Runtime Environment Participants may assume a Python-based runtime environment. 3.14.2 works correctly. DCL Simulation software works on windows 11 with standard PC and GPU with 8 GB VRAM (no linux/mac support)

Client is responsible for

- Establishing MAVLink comms
- Maintaining heartbeat messages
- Sending control commands
- Processing telemetry data
- Processing vision stream data

Example control architecture



Example control session

1. Client initializes MAVSDK
2. Client connects to simulator endpoint
3. Simulator transmits HEARTBEAT
4. Client streams control commands
5. Simulator applies commands

6. Telemetry and vision streams returned

Participants must ensure implementation conforms to specs. No human interaction allowed during runtime.

Round 1 - Qualification

- Objective: verifies contestant software can navigate successfully the race course
- Structure: start gate, intermediate gates, and finish gate
- Maximum runtime of 8 minutes

Email with FAQ from 4/9

Copied the most important seeming sections

Data & Control Interface

- Inputs: FPV visual stream + telemetry
- Outputs: Standard drone control (Throttle, Roll, Pitch, Yaw)
- No GPS or absolute positioning provided
- No depth data or additional sensor shortcuts available
- Camera details will be shared before the physical stage

Rules & Performance

- Unlimited attempts within the qualification window
- Virtual Qualifier 1: Focus on completion
- Virtual Qualifier 2: Fastest valid time counts
- Maximum run time: 8 minutes per attempt
- Gates must be passed in the correct order

Competition Structure

- Virtual Qualifier 1: Short, simplified simulator course (focus on successful completion)
- Virtual Qualifier 2: Longer and more complex simulator course (fastest time counts)
- Physical Qualifier: Real drones, controlled environment (no audience)
- Grand Prix: Real drones under race conditions with environmental distractions and audience
- Technical specifications currently cover only the Virtual Qualifier and will be expanded in future updates

Simulator & Environment Insights

- The simulator will be provided as a downloadable package and runs locally
- An active internet connection is required (anti-cheat measures)
- Windows is supported (Linux is not recommended)
- Teams can run multiple instances in parallel
- Gates remain consistent within each qualifier but change positions between Virtual Qualifier 1 and 2
- Start and finish gate may differ slightly

- Non-gated objects may be present in the environment
- Number of gates: $VQ1 < 10$, $VQ2 < 20$
- Full 3D environment with elevation changes

Questions and Answers

What is an IMU?

An IMU is the drone's "sense of balance and motion," allowing it to understand how it's moving and stay stable in the air.

A typical IMU in a drone combines multiple sensors:

- Accelerometer → measures linear acceleration (movement along x, y, z axes)
- Gyroscope → measures rotational velocity (how fast the drone is turning)
- (Sometimes) Magnetometer → measures orientation relative to Earth's magnetic field (like a compass)

What is cx, cy and fx, fy?

(cx, cy) Image center (principal point) pixels

Since the image is **640×360**, these values are exactly the center at **360×180**. So the image center is the true center.

(fx, fy) Focal length (projection scaling) pixels

These represent the focal length projected into the image plane, expressed in pixels.

They scale 3D coordinates → 2D image coordinates.

Separate values allow for non-square pixels, but here:

$$f_x = f_y = 320$$

These control:

how "zoomed in" the camera is

how wide the field of view is

$$\text{FoV}_y = 2 \cdot \arctan\left(\frac{H}{2f_y}\right)$$

With:

- $H = 360$
- $f_y = 320$

$$\text{FoV}_y \approx 2 \cdot \arctan(360/(2 \cdot 320)) \approx 2 \cdot \arctan(0.5625) \approx 90^\circ$$

Values check out from MS Copilot.